Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

F. Gợi ý hướng giải 2
F.1. Chương 1: Tổng quan SOA & Microservices 2
F.2. Chương 2: Phân tích hướng domain 2
F.3. Chương 3: Thiết kế dịch vụ & API 3
F.4. Chương 4: Giao tiếp đồng bộ 3
F.5. Chương 5: Giao tiếp bất đồng bộ 4
F.6. Chương 6: Giao dịch phân tán 4
F.7. Chương 7: Quản lý dữ liệu 5
F.8. Chương 8: API Gateway & BFF Pattern 5
F.9. Chương 9: Bảo mật Microservices 5
F.10. Chương 10: Chuyển đổi thực tế 6
F.11. Chương 11: Quan sát hệ thống 6
F.12. Chương 12: Triển khai & DevOps 7
F.13. Capstone: Thiết kế “Online Judge” 7

PHỤ LỤC F

Gợi ý hướng giải

Gợi ý hướng giải cho các bài tập trong phần "Bài tập & case study". Gợi ý chỉ định hướng, không phải đáp án đầy đủ: nhiều bài (đặc biệt mức `L2` và `L3`) là bài mở, không có một đáp án đúng duy nhất; giá trị nằm ở quá trình phân tích và lập luận.

F.1. Chương 1: Tổng quan SOA & Microservices

1-0 `NOTE` **Ôn tập** `L1`: A là 2, B là 1, C là 2, D là 3, E là 1, F là 3. Martin Fowler khuyên nên bắt đầu bằng monolith (Monolith First), chỉ tách nhỏ khi có đủ lý do xác đáng. Xem §1.6. Đối với khả năng mở rộng, kiến trúc monolith bắt buộc phải nhân bản toàn bộ hệ thống ngay cả khi chỉ một tính năng bị quá tải, trong khi kiến trúc microservices cho phép chỉ nhân bản dịch vụ chịu tải cao, giúp phân bổ tài nguyên hiệu quả hơn. Dấu hiệu monolith đã sẵn sàng để tách: mỗi thay đổi nhỏ phải phối hợp nhiều đội, chu kỳ phát hành kéo dài, không mở rộng riêng được phần chịu tải (Bảng 1.1). Về SOA: microservices đẩy mạnh Loose Coupling, Autonomy và giảm nhẹ Reusability ở cấp doanh nghiệp, điều phối qua ESB, vì độc lập được ưu tiên hơn tái sử dụng (§1.2).

1-1 `EVAL` `L2`: Công ty A: chưa nên (Not Yet) do đang là startup, chưa đạt độ phù hợp sản phẩm-thị trường (product-market fit). Công ty B: nên chuyển đổi (Yes) vì đã có đủ đội ngũ và điểm nghẽn (pain point) rõ ràng. Công ty C: không bao giờ hoặc chưa nên (Never/Not Yet) vì đây chỉ là hệ thống ERP nội bộ với lượng người dùng rất nhỏ (500 người dùng). Xem Bảng 10.1 về ma trận quyết định (Decision Matrix).

1-2 `CASE` `L3`: Shopify lựa chọn kiến trúc Modular Monolith vì ranh giới giữa các thành phần (component boundary) đã đáp ứng đủ cho mức độ mở rộng (scale) của họ. Xem §1.6.

F.2. Chương 2: Phân tích hướng domain

2-0 `NOTE` **Ôn tập** `L1`: Bounded Context tương ứng với (b); Ubiquitous Language tương ứng với (b); Aggregate tương ứng với (a). "Bài nộp" có ba nghĩa: câu trả lời SQL (SQL Practice), nội dung nộp bài theo rubric (Academic Management), bài thi có hạn giờ (Examination & Proctoring). Xem §2.3. "Tài khoản" thuộc Identity & Access. Các Bounded Context giao tiếp với nhau qua API hoặc

sự kiện. Chuyên gia nghiệp vụ (Domain Expert) bảo đảm mô hình phần mềm phản ánh đúng quy trình thực tế; nếu kỹ sư tự thiết kế một mình, ranh giới Bounded Context và Ubiquitous Language dễ bị xác định sai.

2-1 `DESIGN` `L2`: Luồng sự kiện: yêu cầu đăng ký (RegistrationRequested), kiểm tra điều kiện tiên quyết (PrerequisiteChecked), giữ chỗ thành công (SeatReserved) hoặc vào danh sách chờ (WaitlistJoined), khởi tạo thanh toán (BillingInitiated), thanh toán hoàn tất (PaymentCompleted) hoặc hết hạn thanh toán (PaymentExpired), xác nhận đăng ký (RegistrationConfirmed) hoặc hủy đăng ký (RegistrationCancelled). Các Bounded Context gồm: danh mục môn học, đăng ký, thanh toán và hồ sơ sinh viên. Theo Erl: Task là điều phối đăng ký; Entity là học phần, sinh viên, hóa đơn; Utility là thông báo. Erl đi theo quy trình, DDD phát hiện khác biệt ngôn ngữ (§2.3).

2-2 `EVAL` `L2`: Trước hết phân biệt kiểm tra định dạng ổn định dùng chung với quy tắc nghiệp vụ do một context sở hữu: quy tắc nghiệp vụ (domain logic) cần được giữ lại bên trong dịch vụ, còn thư viện kiểm tra trung lập có thể dùng chung khi lợi ích vượt coupling và có quản lý version. Thư viện dùng chung (shared library) chỉ nên chứa các đối tượng truyền dữ liệu (DTO), mã lỗi (error code) và các nghiệp vụ cắt ngang (cross-cutting concern) như ghi nhật ký (logging). `StudentId` là mô hình miền nên thuộc Shared Kernel, các context đồng sở hữu phải cùng đồng ý (§2.5). Xem Newman [4a].

F.3. Chương 3: Thiết kế dịch vụ & API

3-0 `NOTE` **Ôn tập** `L1`: A là non-breaking (thêm trường dữ liệu); B là breaking (đổi tên đồng nghĩa với xóa trường cũ và thêm trường mới); C là non-breaking (tham số tùy chọn); D là breaking (thay đổi mã trạng thái HTTP); E là breaking (xóa trường); F là breaking (thêm trường bắt buộc). Tolerant Reader là nguyên tắc trong đó consumer tự bỏ qua các trường dữ liệu không nhận diện được. Xem §3.9. Tính tương thích ngược đặc biệt quan trọng trong microservices vì các dịch vụ được triển khai độc lập; phá vỡ hợp đồng dữ liệu sẽ gây lỗi trực tiếp cho các consumer chưa kịp cập nhật. URL path rõ ràng, dễ định tuyến nhưng phải duy trì nhiều URL; custom header giữ URL sạch nhưng khó gọi thử và debug (§3.2). `@RestController` là inbound adapter, `QuestionRepository` là outbound port, `JpaQuestionAdapter` là outbound adapter; `JpaRepository` mang kiểu của Spring Data nên thuộc adapter (§3.5). CRUD dùng REST; dashboard hợp GraphQL ở tầng BFF, cần DataLoader (§3.7).

3-1 `CASE` `L2`: Stripe sử dụng định dạng phiên bản theo ngày (date-based versioning) để đảm bảo tính tương thích ngược dài hạn (backward compatibility). Idempotency Key là một chuỗi định danh (UUID) do client sinh ra và đính kèm vào header của request. Xem §3.1 (Idempotency Key), §3.2 (versioning).

3-2 `DEBUG` `L3`: Nguyên nhân gốc rễ: việc đổi tên trường dữ liệu (tương đương xóa trường cũ và thêm trường mới) khiến các consumer phiên bản cũ gặp lỗi. Phòng ngừa: kiểm thử hợp đồng (contract testing) và áp dụng mẫu Expand-and-Contract (mở rộng rồi thu hẹp). Xem §3.2, §3.6.

F.4. Chương 4: Giao tiếp đồng bộ

4-0 `NOTE` **Ôn tập** `L1`: A là 2, B là 4, C là 1, D là 5, E là 3. Circuit Breaker chuyển từ trạng thái Closed sang Open khi lỗi vượt ngưỡng, sau một khoảng thời gian thì sang Half-Open để thử lại, và quay lại Closed nếu thành công. Xem §4.4. Câu 2: tầng ngoài hết hạn trước thì kết quả của tầng trong bị bỏ phí. Câu 3: tên service được tra trong danh sách instance lấy từ Eureka, rồi client tự chọn một instance. Câu 4: lời gọi vẫn chạy nhờ danh sách đã lưu ở client, nên registry không nằm trên đường đi của mỗi request (§4.3).

4-1 `DEBUG` `L3`: Nguyên nhân gốc rễ: thiếu Circuit Breaker và đặt timeout quá dài (30 giây). Khi có Circuit Breaker, dịch vụ báo lỗi nhanh (fail-fast) thay vì giữ luồng chờ đến hết thời gian timeout. Kỹ thuật Bulkhead giúp giới hạn phạm vi ảnh hưởng (blast radius). Xem §4.4, §4.5.

4-2 `EVAL` `L2`: A chọn GraphQL vì phù hợp cho các truy vấn linh hoạt và băng thông hạn chế; B chọn gRPC vì đáp ứng thông lượng cao và schema dữ liệu nghiêm ngặt; C chọn GraphQL vì phù hợp cho cấu trúc dữ liệu lồng nhau; D chọn REST để đảm bảo tính ổn định và dễ dàng lập tài liệu. Xem §4.1 (và §3.7 cho GraphQL). Về Circuit Breaker trong KBM: đặt theo từng worker ở `kbm-judge-sql`, vì Core chỉ làm việc với một coordinator và không phân biệt được worker nào lỗi (§4.5).

F.5. Chương 5: Giao tiếp bất đồng bộ

5-0 `NOTE` **Ôn tập** `L1`: Thứ tự điền: (1) Topic, (2) Partition, (3) Offset, (4) Consumer Group, (5) Producer, (6) Broker. Luồng xử lý bài nộp (submission pipeline) của KBM nên áp dụng chiến lược chuyển giao ít nhất một lần (at-least-once) kết hợp với idempotent consumer. Xem §5.3, §5.5. `JudgeSubmission` là Command, có thể bị từ chối; `SubmissionJudged` là Event, đã xảy ra nên không từ chối được; `UserDocument` là Document (§5.6). Exchange kiểu Topic so khớp routing key với binding của từng queue (§5.2).

5-1 `EVAL` `L2`: A chọn RabbitMQ vì phù hợp cho định tuyến đơn giản và cơ chế xác nhận; B chọn Kafka vì đảm bảo thứ tự và lưu giữ luồng sự kiện; C chọn Kafka vì đáp ứng thông lượng cao; D chọn RabbitMQ vì điều phối vòng tròn và cơ chế xác nhận. Xem §5.2.

5-2 `DEBUG` `L3`: Poison Pill là thông điệp không thể giải tuần tự hóa (deserialize), khiến consumer rơi vào vòng lặp lỗi (crash loop). Giải pháp: dùng `ErrorHandlingDeserializer` để lỗi giải tuần tự hóa đi thẳng sang Dead Letter Topic, vì thử lại không sửa được dữ liệu hỏng; retry có backoff chỉ dành cho lỗi tạm thời; sau đó rà soát thủ công. Xem §5.9.

F.6. Chương 6: Giao dịch phân tán

6-0 `NOTE` **Ôn tập** `L1`: Thứ tự thực hiện: (1) Core tạo Submission, (2) Judge chấm bài, (3) Ghi điểm Gradebook, (4) Ghi bền vững lệnh thông báo vào outbox; notification worker gửi email ngoài saga với retry giới hạn và đối soát khi thất bại. Phân loại: bước 1 là Compensatable, bước 2 là Pivot, bước 3 và 4 là Retriable (§6.2). Compensation cho bước 1 là đánh dấu Submission là ERROR (C1 — Mark ERROR). Saga *không* đảm bảo ACID, nó đảm bảo ACD (tính nguyên tử qua compensation, tính nhất quán giữ bởi quy tắc nghiệp vụ trong mỗi local transaction, và tính bền vững — durability). Lưu ý: "nguyên tử qua compensation" là nguyên tử *nghiệp vụ* (hoàn tất toàn chuỗi hoặc bù trừ các bước đã làm), không phải rollback ACID — trạng thái trung gian vẫn có thể bị quan sát, và mỗi local transaction bên trong vẫn là ACID. Xem §6.2, §6.4.

6-1 `DESIGN` `L3`: Bước 1–3 (đặt chuyến, ghép tài xế, nhận chuyến) là Compensatable. Bước 4 (bắt đầu chuyến) là Pivot: chuyến đã chạy thì không hoàn tác được, như đề đã nêu. Bước 5–9 là Retriable: bước 7 thất bại thì giữ hồ sơ chuyến và chờ thử lại, không bù trừ chuyến. Orchestration phù hợp hơn vì luồng nghiệp vụ phức tạp và đòi hỏi nhiều kịch bản bù trừ (compensation). Xem §6.3.

6-2 `EVAL` `L2`: Mô hình Choreography đơn giản và có liên kết lỏng (loose coupling) nhưng khó truy vết (trace). Mô hình Orchestration cung cấp quản lý tập trung và dễ truy vết hơn, nhưng phải vận hành thêm một thành phần trung tâm và có nguy cơ gom quá nhiều nghiệp vụ vào đó. Hướng lập luận: A đơn giản nên choreography là đủ; B cần giám sát thanh toán nên nghiêng về orchestration;

C có con người tham gia, kéo dài nhiều ngày, và D có trạng thái không chắc chắn với API ngoài, nên đều cần orchestrator giữ trạng thái. Xem §6.3.

F.7. Chương 7: Quản lý dữ liệu

7-0 `NOTE` **Ôn tập** `L1`: A là 3, B là 2, C là 1, D là 4. Theo định lý CAP, chỉ khi xảy ra network partition (phân vùng mạng), hệ thống mới phải đánh đổi giữa tính nhất quán (Consistency) và tính sẵn sàng (Availability); khi không có partition, hệ thống có thể bảo đảm cả C lẫn A. Vấn đề khi KBM dùng chung cơ sở dữ liệu là các dịch vụ bị ràng buộc về schema (schema coupling), không thể mở rộng hay triển khai độc lập. Xem §7.1.

7-1 `EVAL` `L3`: Cách nói "chọn 2 trong 3" của định lý CAP là sự hiểu lầm, vì khả năng chịu phân vùng mạng (partition tolerance — ký hiệu P) không phải là một tùy chọn: network partition luôn có thể xảy ra trong hệ thống phân tán. Chỉ khi network partition thực sự xảy ra, hệ thống mới phải chọn giữa C và A. Xem §7.1.

7-2 `DESIGN` `L2`: Bước 1 là tách schema (`core_schema`, `academic_schema`), truy vấn chéo còn lại tạm đọc qua view. Bước 2 là thay truy vấn chéo bằng API rồi thu hồi quyền đọc chéo. Bước 3 là nhân bản dữ liệu tham chiếu qua sự kiện. Tách hẳn database server chỉ khi cần mở rộng độc lập. Xem §7.2 (Áp dụng cho KBM).

F.8. Chương 8: API Gateway & BFF Pattern

8-0 `NOTE` **Ôn tập** `L1`: Thuộc Gateway gồm xác minh JWT ở biên (edge token validation / auth), rate limiting, routing, SSL termination, load balancing và response caching. Không thuộc Gateway gồm business logic, database migration và tính điểm bài tập (grading logic). Xem §8.4.

8-1 `CASE` `L2`: Netflix chuyển từ Zuul 1 (blocking, mỗi request một thread) sang Zuul 2 (non-blocking, dựa trên Netty) và vận hành Zuul 2 ở quy mô lớn. Spring Cloud Gateway là dự án riêng của Spring, xây trên WebFlux (non-blocking), thay thế Spring Cloud Netflix Zuul. Cái giá của non-blocking: luồng xử lý khó theo dõi, gỡ lỗi khó hơn, và mọi thành phần trên đường đi của request đều phải bất đồng bộ. Xem §8.2.

8-2 `ADR` `L2`: Phương án 1 nhắm đúng người lạm dụng nhờ định danh đã xác thực. Phương án 2 dễ chặn nhầm cả phòng máy đi ra qua một IP NAT (§8.4.4). Phương án 3 bảo vệ đúng endpoint thi. Phương án 4 phức tạp so với nhóm nhỏ. Có thể chọn kết hợp 1 và 3. Xem §8.4.

F.9. Chương 9: Bảo mật Microservices

9-0 `NOTE` **Ôn tập** `L1`: A là AuthN; B là AuthZ; C là AuthN; D là AuthZ; E là AuthZ (OAuth 2.0 cấp quyền truy cập, không xác thực danh tính); F là AuthZ. JWT gồm Header (algorithm), Payload (claim), và Signature. RS256 cho phép xác minh (verify) mà không cần bí mật chung (shared secret), điều này quan trọng khi có nhiều dịch vụ. Xem §9.2, §9.5. Phân cấp vai trò là ADMIN > LECTURER > STUDENT; quyền "sửa bài nộp của chính mình" cần quan hệ sở hữu, chỉ xét vai trò thì không đủ (§9.5).

9-1 `DEBUG` `L3`: Khi phát hiện token bị đánh cắp (token theft), hệ thống cần áp dụng cơ chế danh sách đen (token blacklist), sử dụng access token có vòng đời ngắn (short-lived access token) kết hợp với cơ chế luân phiên (rotation) cho refresh token. Xem §9.2.

9-2 `EVAL` `L2`: A (React SPA) và D (ứng dụng di động React Native) dùng luồng Authorization Code kèm PKCE, vì cả hai là client công khai, không giữ được client secret an toàn. B (công cụ CLI không có trình duyệt, quản trị viên đăng nhập) dùng Device Authorization Grant. C (Judge Service, giao tiếp máy với máy, không có người dùng) dùng luồng Client Credentials. Ứng dụng web có backend riêng trên máy chủ, không có trong đề, dùng luồng Authorization Code chuẩn. Xem §9.4.

F.10. Chương 10: Chuyển đổi thực tế

10-0 `NOTE` **Ôn tập** `L1`: Thứ tự là B, C, E, A, D. Các anti-pattern gồm Big Bang rewrite (viết lại toàn bộ một lần) và Distributed Monolith. Chiến lược đập bỏ xây mới (Big Bang) là sai lầm do rủi ro quá cao; thay vào đó nên áp dụng mẫu thiết kế Strangler Fig để bóc tách từng phần. Với KBM, ứng viên là phân hệ thi (`kbm-exam`), vì logic Examination đã có ranh giới nghiệp vụ rõ; cần làm rõ Bounded Context trước rồi mới chuyển hướng định tuyến. Xem §10.2, Phụ lục D.

10-1 `CASE` `L3`: Sự kiện API Mandate tại Amazon năm 2002 bắt buộc mọi nhóm phát triển phải bộc lộ chức năng thông qua API (expose service API). Quá trình chuyển đổi này thực tế đã kéo dài hơn 10 năm. Câu 2: dùng các bước ở §10.2. Câu 3: xem Conway's Law ở Chương 2.

10-2 `EVAL` `L2`: Outbox Pattern đơn giản vì dùng giao dịch cục bộ của cơ sở dữ liệu; Event Sourcing tạo ra lưu vết lịch sử (audit trail) nhưng có độ phức tạp cao hơn; mô hình CDC can thiệp gián tiếp (không xâm lấn mã nguồn, non-invasive) nhưng yêu cầu công cụ bên thứ ba như Debezium. Xem §10.3.

10-3 `DESIGN` `L3`: Về ma trận: log có cấu trúc và Circuit Breaker có tác động cao, nỗ lực thấp nên làm trước; tách schema và adapter ACL tốn nỗ lực hơn; rate limit theo kỳ thi làm khi cần cho chế độ thi. Về đồng bộ: (A) khó cấu hình các trigger trên cơ sở dữ liệu và làm ẩn đi logic nghiệp vụ; (B) dễ lập trình nhưng gây liên kết chặt (tight coupling) vì monolith liên tục gọi đồng bộ đến API sẽ tạo ra điểm nghẽn; (C) hàng đợi sự kiện (Event Queue) là phương án có liên kết lỏng nhất (loose coupling) vì monolith chỉ cần lắng nghe sự kiện AttendanceRecorded để tự động cập nhật điểm. Rủi ro chung là phải duy trì logic ở hai bộ mã nguồn, do đó cần đóng băng phần logic cũ trong monolith sớm nhất có thể.

F.11. Chương 11: Quan sát hệ thống

11-0 `NOTE` **Ôn tập** `L1`: A là Metrics; B là Traces; C là Logs; D là Metrics; E là Traces; F là Logs. SLI là chỉ số đo lường thực tế (ví dụ độ trễ P99); SLO là mục tiêu nội bộ hệ thống cần đạt (ví dụ P99 dưới 500 ms); SLA là bản cam kết pháp lý với khách hàng. Lỗi 503 theo RFC 9457 có `type`, `title`, `status`, `detail`; thêm `traceId` để đối chiếu với log (§11.5). Cảnh báo nên được thiết lập dựa trên SLO, vì nếu chỉ số phần cứng (CPU) cao nhưng trải nghiệm người dùng chưa bị ảnh hưởng thì chưa cần thiết phải báo động. Xem §11.1, §11.4.

11-1 `DEBUG` `L3`: 8000 ms nằm giữa Kafka publish và Judge receive: message đang chờ trong hàng đợi, Judge không chậm khi đã nhận bài. Một consumer thread xử lý tuần tự 120 bài nên lag tăng. Thêm instance chỉ có tác dụng khi số partition tăng tương ứng (§5.9). Xem §11.3, §11.4.

11-2 `EVAL` `L2`: Tự vận hành mã nguồn mở (A, Self-hosted OSS) linh hoạt nhưng tốn công sức vận hành. Dịch vụ được quản lý cho mã nguồn mở (B, Managed OSS, như Grafana Cloud) giảm gánh nặng hạ tầng, chi phí theo mức sử dụng. Giải pháp thương mại (C, như Datadog hoặc New Relic) thiết lập nhanh, có hỗ trợ, nhưng chi phí cao. Công cụ gốc của nhà cung cấp đám mây (D, như

AWS CloudWatch) tiện lợi khi hệ thống đã chạy trên đám mây đó, nhưng có rủi ro phụ thuộc nhà cung cấp (vendor lock-in). Xem §11.1–11.3, §11.9.

F.12. Chương 12: Triển khai & DevOps

12-0 `NOTE` **Ôn tập** `L1`: A là 2 (Blue-Green), B là 1 (Canary), C là 4 (Rolling), D là 3 (Shadow). Container chia sẻ nhân hệ điều hành (OS kernel) nên khởi động nhanh; máy ảo (VM) có hệ điều hành riêng nên cô lập chặt chẽ hơn. Infrastructure as Code (IaC) đảm bảo mọi cấu hình đều được viết bằng mã, lưu trữ phiên bản và có tính lặp lại (reproducible). Đối với hệ thống KBM triển khai bằng Docker Compose đơn instance, thực chất chỉ có chiến lược All-at-once (Recreate; chấp nhận gián đoạn ngắn, chọn khung giờ thấp điểm) hoặc Blue-Green thủ công sau reverse proxy nếu cần triển khai không gián đoạn (zero-downtime); Rolling Update đúng nghĩa đòi hỏi orchestrator điều phối từng replica (Swarm/Kubernetes). Xem §12.2, §12.5.

12-1 `EVAL` `L2`: Docker Compose phù hợp cho môi trường phát triển, môi trường staging hoặc các hệ thống quy mô nhỏ. Kubernetes là lựa chọn hợp lý khi hệ thống cần tự động mở rộng (auto-scaling), tự phục hồi (self-healing) và có nhiều dịch vụ. Xem §12.6.

12-2 `DESIGN` `L3`: Pipeline CI/CD gồm các bước: gửi mã nguồn (commit), build, kiểm thử, triển khai lên staging, phê duyệt thủ công, phát hành canary lên production và phát hành toàn bộ (full rollout). Việc phát hiện service có thay đổi kết hợp kiểm tra khác biệt mã nguồn (`git diff`) với đồ thị phụ thuộc (dependency graph). Xem §12.4.

F.13. Capstone: Thiết kế “Online Judge”

Capstone `L3`: Đề xuất 4 dịch vụ tối thiểu: API Gateway, Submission Service (nộp bài), Judge Service (chấm bài) và Leaderboard Service (bảng xếp hạng). Giao tiếp: sử dụng đồng bộ (REST hoặc gRPC) cho hành động nộp bài và bất đồng bộ (Kafka) cho hàng đợi chấm điểm. Dữ liệu: áp dụng kiến trúc CQRS cho bảng xếp hạng (sử dụng Redis Sorted Set cho mô hình đọc). Tính chịu lỗi (resilience): nêu rõ ranh giới lỗi của từng luồng — Circuit Breaker khi có lời gọi đồng bộ cần bảo vệ; retry, timeout và Dead Letter Topic (DLT) cho luồng message tới dịch vụ chấm bài. Khả năng mở rộng (scale): nhân bản theo chiều ngang (horizontal scale) dịch vụ chấm bài, đồng thời chia partition Kafka theo `userId` để giữ thứ tự theo sinh viên (§5.9) — tránh key theo ID kỳ thi vì sẽ dồn cả kỳ thi vào một partition.